



CENTRO UNIVERSITARIO DE JOÃO PESSOA – UNIPÊ
CURSO DE GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO

HELLEN LEITE DOS SANTOS
ANA JULIA FERNANDES BARRETO
BRUNA EDILEUZA FERREIRA CAVALCANTE
VITORIA MYRELLA MOURA NONATO
DANIEL ASAPH DA SILVA FARIAS
LUAN SILVEIRA SOARES GERÔNICO

TRABALHO: PROJETO ETAPA 2 A CLINICA VIDA PLENA

JOÃO PESSOA

JUNHO 2026

1. Introdução

O sistema VidaPlena foi desenvolvido com o propósito de auxiliar na administração de uma clínica multidisciplinar, reunindo em uma única aplicação funcionalidades relacionadas ao gerenciamento de pacientes, profissionais de saúde, consultas, atendimentos e pagamentos.

A proposta do projeto foi simular situações comuns do cotidiano de uma clínica, permitindo o cadastro de pacientes e especialistas, o controle de horários disponíveis, o agendamento e acompanhamento de consultas, além do registro de informações clínicas por meio de prontuários. O sistema também contempla diferentes formas de pagamento e mecanismos para exportação de dados e geração de relatórios.

Durante o desenvolvimento, foram aplicados diversos conceitos de Programação Orientada a Objetos, buscando não apenas atender aos requisitos funcionais, mas também construir uma solução organizada, reutilizável e de fácil manutenção. Para isso, foram utilizados recursos como encapsulamento, herança, abstração, polimorfismo e interfaces, além de diferentes tipos de relacionamentos entre objetos.

As principais funcionalidades implementadas incluem:

- Cadastro e gerenciamento de pacientes;
- Cadastro de profissionais de diferentes especialidades;
- Controle de disponibilidade de horários;
- Agendamento, cancelamento e remarcação de consultas;
- Registro de atendimentos clínicos;
- Criação e manutenção de prontuários;
- Processamento de pagamentos por diferentes modalidades;
- Exportação de informações do sistema;
- Emissão de relatórios gerenciais.

2. Diagrama de Classes

2.1 Diagrama UML em Mermaid

```
classDiagram
```

```
class Pessoa{  
    <<abstract>>  
    #nome  
    #cpf  
    #telefone  
    #idade  
}
```

```
class Paciente{  
    -convenio  
    -ativo
```

```
}
```

```
class Profissional{  
    -especialidade  
    -registroProfissional  
    -valorConsulta  
    -horariosDisponiveis  
    +registrarEspecifico()  
    +exibirResumo()  
}
```

```
class ClinicoGeral  
class Psicologo  
class Nutricionista  
class Fisioterapeuta
```

```
class HorarioDisponivel{  
    -diaSemana  
    -turno  
}
```

```
class Consulta{  
    -cpfPaciente  
    -nomeProfissional  
    -data  
    -horario  
    -tipo  
    -status  
    +agendar()  
    +cancelar()  
    +remarcar()  
}
```

```
class Atendimento{  
    -consulta  
    -prontuario  
}
```

```
class Prontuario{  
    -observacoes  
    -diagnosticoDoPaciente  
    -procedimentosRealizados  
    -dataCriacao  
}
```

```
class Convenio{  
    -nome  
    -percentualCobertura  
}
```

```
class Pagamento{  
    <<abstract>>  
    #consulta  
    #valorFinal  
    #tipoPagamento  
    #parcelas  
    +calcularValorFinal()  
}
```

```
class PagamentoDinheiro
```

```

class PagamentoCartao
class PagamentoConvenio

class Relatorio

class Agendavel{
<<interface>>
agendar()
cancelar()
remarcar()
}

class Exportavel{
<<interface>>
exportarDados()
}

Pessoa <|-- Paciente
Pessoa <|-- Profissional

Profissional <|-- ClinicoGeral
Profissional <|-- Psicologo
Profissional <|-- Nutricionista
Profissional <|-- Fisioterapeuta

Pagamento <|-- PagamentoDinheiro
Pagamento <|-- PagamentoCartao
Pagamento <|-- PagamentoConvenio

Consulta ..|> Agendavel
Consulta ..|> Exportavel

Atendimento ..|> Exportavel
Pagamento ..|> Exportavel

Paciente --> Convenio
PagamentoConvenio --> Convenio

Profissional o-- HorarioDisponivel

Atendimento *-- Prontuario
Atendimento --> Consulta
Pagamento --> Consulta

```

2.2 Descrição do Diagrama

A classe abstrata Pessoa representa a entidade base do sistema, armazenando informações comuns a pacientes e profissionais.

Paciente herda de Pessoa e adiciona informações relacionadas ao convênio e ao status de atividade do paciente.

Profissional também herda de Pessoa e serve como base para as especialidades médicas implementadas: Clínico Geral, Psicólogo, Nutricionista e Fisioterapeuta.

Consulta representa os agendamentos realizados pelos pacientes e implementa a interface Agendavel.

Atendimento representa a execução de uma consulta e possui um prontuário próprio, caracterizando uma relação de composição.

Pagamento é uma classe abstrata responsável por centralizar o processamento financeiro, possuindo três especializações para diferentes formas de pagamento.

A interface Exportavel padroniza a exportação de dados para diferentes entidades do sistema.

3. Mapa de Aplicação dos Conceitos de POO

3.1 Encapsulamento

Classes envolvidas

- Prontuario
- HorarioDisponivel
- Profissional
- Paciente
- Consulta

Aplicação

Os atributos foram declarados como privados e acessados por meio de getters e setters.

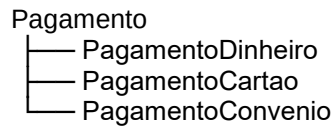
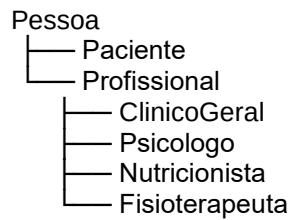
Exemplo:

```
private String observacoes;  
  
public String getObservacoes() {  
    return observacoes;  
}  
  
public void setObservacoes(String observacoes) {  
    this.observacoes = observacoes;  
}
```

O encapsulamento protege o estado interno dos objetos e permite validações antes da alteração dos dados.

3.2 Herança

Classes envolvidas



A herança foi utilizada para evitar duplicação de código e centralizar características comuns.

3.3 Polimorfismo

Sobrescrita de métodos

Especialidades médicas:

```
@Override  
public void registrarEspecifico(Atendimento atendimento)
```

Método implementado de forma diferente em:

- ClinicoGeral
- Psicologo
- Nutricionista
- Fisioterapeuta

Pagamentos:

```
@Override  
public double calcularValorFinal()
```

Implementado de maneira distinta em:

- PagamentoDinheiro
- PagamentoCartao
- PagamentoConvenio

O comportamento varia conforme o tipo concreto do objeto utilizado.

3.4 Abstração

Classe Pessoa

```
public abstract class Pessoa
```

Representa características genéricas compartilhadas entre pacientes e profissionais.

Classe Pagamento

```
public abstract class Pagamento
```

Define a estrutura comum para todos os tipos de pagamento.

3.5 Interfaces

Interface Agendavel

```
public interface Agendavel
```

Implementada por:

- Consulta

Métodos:

- agendar()
- cancelar()
- remarcar()

Interface Exportavel

Implementada por:

- Consulta
- Atendimento
- Pagamento

Método:

```
String exportarDados();
```

3.6 Associação

Paciente → Convênio

Um paciente pode possuir um convênio.

private Convenio convenio;

Ambos os objetos existem independentemente.

PagamentoConvenio → Convenio

O pagamento utiliza informações do convênio para calcular descontos.

3.7 Agregação

Profissional → HorarioDisponivel

private List<HorarioDisponivel> horariosDisponiveis;

Os horários podem existir independentemente do profissional.

3.8 Composição

Atendimento → Prontuario

private Prontuario prontuario;

O prontuário é criado dentro do atendimento e depende totalmente dele para existir.

4. Jornadas de Usuário da AV1

Jornada 1 – Cadastro de Paciente

Objetivo:

Cadastrar um novo paciente no sistema.

Passos:

1. Informar nome.
2. Informar CPF.
3. Informar telefone.
4. Informar idade.
5. Selecionar convênio, quando houver.
6. Confirmar cadastro.

Resultado esperado:

Paciente registrado e disponível para futuras consultas.

Jornada 2 – Cadastro de Profissional

Objetivo:

Cadastrar um profissional da clínica.

Passos:

1. Informar dados pessoais.
2. Informar especialidade.
3. Registrar número profissional.
4. Definir valor da consulta.
5. Definir horários disponíveis.

Resultado esperado:

Profissional disponível para agendamentos.

Jornada 3 – Agendamento de Consulta

Objetivo:

Marcar uma consulta para um paciente.

Passos:

1. Selecionar paciente.
2. Selecionar profissional.
3. Escolher data.
4. Escolher horário.
5. Confirmar agendamento.

Resultado esperado:

Consulta criada com status "Agendada".

Jornada 4 – Cancelamento de Consulta

Objetivo:

Cancelar uma consulta previamente marcada.

Passos:

1. Localizar consulta.
2. Solicitar cancelamento.
3. Registrar motivo opcional.

Resultado esperado:

Status alterado para "Cancelada".

Jornada 5 – Remarcação de Consulta

Objetivo:

Alterar data ou horário.

Passos:

1. Selecionar consulta.
2. Escolher nova data.
3. Escolher novo horário.
4. Confirmar alteração.

Resultado esperado:

Status alterado para "Remarcada".

5. Novas Jornadas da AV2

Jornada 1 – Registro de Atendimento

Classes envolvidas:

- Atendimento
- Consulta
- Prontuario

Passos:

1. Selecionar consulta realizada.
2. Inserir observações.
3. Registrar diagnóstico.
4. Registrar procedimentos.

Resultado esperado:

Atendimento registrado e prontuário criado.

Jornada 2 – Registro Específico por Especialidade

Classes envolvidas:

- ClinicoGeral
- Psicologo

- Nutricionista
- Fisioterapeuta

Passos:

1. Realizar atendimento.
2. Executar registrarEspecifico().
3. Inserir informações da especialidade.

Resultado esperado:

Prontuário enriquecido com dados especializados.

Jornada 3 – Processamento de Pagamento

Classes envolvidas:

- Pagamento
- PagamentoDinheiro
- PagamentoCartao
- PagamentoConvenio

Passos:

1. Selecionar consulta.
2. Escolher forma de pagamento.
3. Calcular descontos ou juros.
4. Confirmar pagamento.

Resultado esperado:

Valor final calculado corretamente.

Jornada 4 – Exportação de Dados

Classes envolvidas:

- Consulta
- Atendimento
- Pagamento

Passos:

1. Selecionar registro.
2. Acionar exportação.
3. Gerar texto estruturado.

Resultado esperado:
Dados exportados para integração ou relatório.

Jornada 5 – Geração de Relatórios

Classes envolvidas:

- Relatorio
- Consulta
- Atendimento

Passos:

1. Selecionar tipo de relatório.
2. Filtrar profissional quando necessário.
3. Processar registros.
4. Exibir informações consolidadas.

Resultado esperado:
Relatório gerencial gerado.

6. Tabela de Relacionamento entre Requisitos e Classes

Funcionalidade	Classes Envolvidas	Conceitos Utilizados
Cadastro de Pacientes	Paciente, Pessoa	Herança, Encapsulamento
Cadastro de Profissionais	Profissional e especializações	Herança, Polimorfismo
Controle de Horários	Profissional, HorarioDisponivel	Agregação
Agendamento de Consultas	Consulta, Agendavel	Interface, Encapsulamento
Atendimento Clínico	Atendimento, Prontuario	Composição
Registro de Diagnósticos	Atendimento, Prontuario	Encapsulamento
Pagamento em Dinheiro	PagamentoDinheiro	Herança, Polimorfismo
Pagamento em Cartão	PagamentoCartao	Herança, Polimorfismo
Pagamento por Convênio	PagamentoConvenio, Convenio	Associação
Exportação de Dados	Consulta, Atendimento, Pagamento	Interface
Relatórios	Relatorio	Reutilização e Modularização

7. Conclusão

O desenvolvimento do sistema permitiu aplicar de forma prática os principais conceitos de Programação Orientada a Objetos estudados na disciplina. A utilização de classes abstratas possibilitou a definição de estruturas genéricas reutilizáveis, enquanto a herança reduziu a duplicação de código entre entidades semelhantes.

O polimorfismo foi empregado principalmente nas especializações dos profissionais de saúde e nas diferentes modalidades de pagamento, permitindo comportamentos distintos a partir de uma mesma estrutura base. As interfaces contribuíram para a padronização de funcionalidades compartilhadas, como agendamento e exportação de dados.

Os relacionamentos de associação, agregação e composição foram utilizados de acordo com o nível de dependência entre os objetos, tornando o modelo mais fiel ao domínio de uma clínica de saúde. A composição entre Atendimento e Prontuário é um exemplo de forte vínculo entre entidades, enquanto a agregação entre Profissional e HorárioDisponível demonstra independência de ciclo de vida.

A modelagem adotada resultou em uma arquitetura organizada, extensível e alinhada aos requisitos funcionais do projeto. Além de atender às necessidades propostas pela disciplina, a solução estabelece uma base sólida para futuras evoluções do sistema, como persistência em banco de dados, interfaces gráficas e integração com serviços externos.